

УЧЕБНАЯ ШПАРГАЛКА

Диагностика внешнего доступа к приложению в Kubernetes

Практический маршрут от симптома «curl не отвечает» до точной причины. Примеры основаны на Managed Kubernetes и Network Load Balancer в Yandex Cloud.

Главная идея

Не проверяй всю систему одним запросом. Раздели путь трафика на слои и двигайся изнутри наружу. Каждый успешный тест исключает уже проверенную часть.

Клиент

- > Public Load Balancer :80
- > Worker node :30080
- > Kubernetes Service
- > Pod :8080
- > Приложение

Правило диагностики

Если запрос к Pod не работает, Load Balancer пока не проверяем. Если Pod и Service отвечают, а внешний IP нет, проблема находится в NodePort, health-check или security group.

Какие адреса участвуют

Пример	Что это	Где доступен
84.252.134.120	Публичный IP Load Balancer	Из интернета
10.10.11.17	Приватный IP рабочей ноды	В VPC
10.96.254.7	Виртуальный ClusterIP Service	Внутри кластера
10.112.128.23	Временный IP Pod	Внутри кластера

1. Проверить рабочие ноды

```
kubectl get nodes -o wide
```

Ожидаем минимум две ноды со статусом Ready. Если нода NotReady, сначала диагностируем её: остальные уровни зависят от рабочих нод.

2. Проверить Pod

```
kubectl get pods \
-n bulletin-board-prod \
-o wide
```

Поле	Что проверяем
READY	Контейнер готов принимать трафик, например 1/1
STATUS	Ожидаем Running
RESTARTS	Счётчик не должен постоянно расти
IP	Внутренний адрес Pod для диагностики
NODE	На какой рабочей ноде запущен Pod

Подробности и события Pod

```
kubectl describe pod \
-n bulletin-board-prod \
<POD_NAME>
```

В разделе Events ищем ошибки скачивания образа, readiness/liveness probe, нехватку ресурсов и проблемы планирования.

3. Проверить логи приложения

```
kubectl logs \
-n bulletin-board-prod \
deployment/bulletin-board-deployment \
--tail=200
```

Быстрый фильтр

```
kubectl logs -n bulletin-board-prod \
deployment/bulletin-board-deployment | \
grep -Ei 'started|8080|9090|error|exception|tomcat|netty'
```

Важный нюанс

Readiness probe на порту 9090 подтверждает работу health endpoint, но не доказывает, что основное приложение отвечает на 8080.

4. Проверить Service

```
kubectl get service \
-n bulletin-board-prod \
backend-service
```

Запись 80:30080/TCP означает: внешний порт Service 80 связан с NodePort 30080. Параметр targetPort в манифесте направляет запрос дальше на порт Pod 8080.

LoadBalancer:80 -> Node:30080 -> Pod:8080

События Service

```
kubectl describe service \
-n bulletin-board-prod \
backend-service
```

Смотрим Events внизу. PermissionDenied указывает на права облачного сервисного аккаунта; EnsuredLoadBalancer означает успешное создание балансировщика. Старые события остаются в истории, поэтому учитывай время.

5. Проверить, нашёл ли Service поды

```
kubectl get endpointslice \
-n bulletin-board-prod \
-l kubernetes.io/service-name=backend-service
```

Endpoint должен содержать IP Pod и порт 8080. Если адресов нет, сравни selector Service с labels Pod:

```
kubectl get pods -n bulletin-board-prod --show-labels
```

**Если endpoints
пуст**

Service не нашёл подходящие Pod. Значения selector и labels должны совпадать буквально.

6. Проверить Service изнутри кластера

```
CLUSTER_IP=$(kubectl get service \
-n bulletin-board-prod \
backend-service \
-o jsonpath='{.spec.clusterIP}')

echo "$CLUSTER_IP"
```

```
kubectl run curl-test \
-n bulletin-board-prod \
--rm -it --restart=Never \
--image=curlimages/curl \
-- curl -v --max-time 10 "http://${CLUSTER_IP}/"
```

Команда создаёт временный Pod с curl в том же namespace, выполняет запрос и удаляет тестовый Pod после завершения.

7. Проверить Pod напрямую

```
POD_IP=$(kubectl get pods \
-n bulletin-board-prod \
-l app=bulletin-board \
-o jsonpath='{.items[0].status.podIP}')
```

```
kubectl run curl-test \
-n bulletin-board-prod \
--rm -it --restart=Never \
--image=curlimages/curl \
-- curl -v --max-time 10 "http://${POD_IP}:8080/"
```

Результат	Вывод
Pod отвечает, ClusterIP нет	Проверить Service и сетевые правила Service
Оба отвечают	Внутренняя часть исправна
Pod не отвечает	Проверить приложение или сеть между Pod

IP Pod временный. Его используют только для диагностики, но не сохраняют в конфигурации.

8. Проверить приложение через port-forward

```
kubectl port-forward \
-n bulletin-board-prod \
deployment/bulletin-board-deployment \
18080:8080
```

Оставь первый терминал открытым. Во втором терминале:

```
curl -v --connect-timeout 5 --max-time 10 \
  http://127.0.0.1:18080/
```

Этот тест обходит Load Balancer и Service. Если он работает, приложение точно слушает 8080.

9. Проверить внешний IP

```
EXTERNAL_IP=$(kubectl get service \
  -n bulletin-board-prod \
  backend-service \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
```

```
curl -v --connect-timeout 5 --max-time 10 \
  "http://${EXTERNAL_IP}"
```

URL в терминале

Используй обычный URL без Markdown-скобок: `curl http://84.252.134.120/`

10. Проверить здоровье целей Load Balancer

```
yc load-balancer network-load-balancer list
```

```
yc load-balancer network-load-balancer get <LOAD_BALANCER_ID>
```

```
yc load-balancer network-load-balancer target-states \
  <LOAD_BALANCER_ID> \
  --target-group-id=<TARGET_GROUP_ID>
```

Статус	Значение
HEALTHY	Нода прошла проверку и получает трафик
UNHEALTHY	Проверка не проходит; пользовательский трафик не направляется
INITIAL	Проверка ещё настраивается
INACTIVE	Балансировщик или цель не активны

11. Проверить security group

```
terraform state show yandex_vpc_security_group.k8s_sg
```

NodePort для внешнего трафика

```
ingress {
  protocol = "TCP"
  port     = 30080
  v4_cidr_blocks = ["0.0.0.0/0"]
}
```

Связь Pod и Service

```
ingress {
  description = "Traffic between Kubernetes pods and services"
  protocol    = "ANY"
  from_port   = 0
  to_port     = 65535
  v4_cidr_blocks = ["10.96.0.0/16", "10.112.0.0/16"]
}
```

Проверки Network Load Balancer

```
ingress {
  description = "Network Load Balancer health checks"
  protocol    = "TCP"
  from_port   = 0
  to_port     = 65535
  predefined_target = "loadbalancer_healthchecks"
}
```

Как читать результат curl

Результат	Что означает	Куда смотреть
Timeout	Пакет не получил ответа	Маршрут, security group, firewall
Connection refused	Адрес доступен, порт не слушается	Порт контейнера, процесс приложения
Could not resolve host	DNS не преобразовал имя в IP	CoreDNS, имя и namespace Service
HTTP 404	Сеть и сервер работают, маршрута нет	URL приложения
HTTP 500	Запрос дошёл, приложение упало при обработке	Логи приложения и зависимости
HTTP 200	Запрос успешно обработан	Переходить к следующему уровню

Безопасный цикл Terraform

```
export YC_TOKEN="$(yc iam create-token)"
terraform fmt
terraform validate
terraform plan -lock-timeout=1m \
  -var-file=secrets.postgres.tfvars \
  -var-file=secret.backend.tfvars
```

Проверь план. Не подтверждай изменения с destroy или must be replaced, если не понимаешь их причину. Только после проверки запускай apply и дождись освобождения state lock.

State lock

Terraform сам создаёт и снимает блокировку. Не запускай параллельные apply, не используй -lock=false и не делай force-unlock, пока активен другой процесс Terraform.

Короткий алгоритм

1. kubectl get nodes
2. kubectl get pods -o wide
3. kubectl logs / kubectl describe pod
4. kubectl get / describe service
5. kubectl get endpointslice
6. curl напрямую к Pod из тестового Pod
7. curl к ClusterIP из тестового Pod
8. yc ... target-states
9. terraform state show security group
10. curl к External IP

Граница сбоя	Вероятная область
Pod не отвечает	Приложение или сеть Pod

Граница сбоя	Вероятная область
Pod отвечает, Service нет	Selector, endpoints или сеть Service
Service отвечает, External IP нет	Load Balancer, NodePort, health-check, security group
Всё отвечает	Внешний доступ настроен

Официальные источники

Yandex Cloud - настройка security groups Managed Kubernetes:

<https://yandex.cloud/ru/docs/managed-kubernetes/operations/connect/security-groups>

Yandex Cloud - проверки состояния Network Load Balancer:

<https://yandex.cloud/ru/docs/network-load-balancer/concepts/health-check>

Yandex Cloud - проверка target states:

<https://yandex.cloud/ru/docs/network-load-balancer/operations/check-resource-health>